
CIS 678 - Machine Learning

– From a Problem to Generalization –

— Instructor: Yong Zhuang —

yong.zhuang@gvsu.edu

Last Class

- Supervised Learning
 - Regression
 - Classification
- Unsupervised Learning
 - Clustering
 - Anomaly detection
- Other Learning Settings
 - Semi-Supervised Learning
 - Self-Supervised Learning
 - Reinforcement Learning
 - Active Learning
 - Collaborative Filtering



Can you identify the learning paradigm from an application?

Example	Learning Paradigm
Predict the selling price of a house	
Group social-network users by their interests	
Selecting medical images for labeling	
Learning to play a game	

Agenda

- Why Learn from Data?
 - Traditional programming and machine learning
- Instance-based and model-based learning
- How Model-Based Learning Works
 - Model selection, training, and prediction
- Generalization: Underfitting and overfitting



Does better training performance always mean a better model?

A Rule-Based Spam Filter

Examine Spam



Write Rules



Apply the Rules



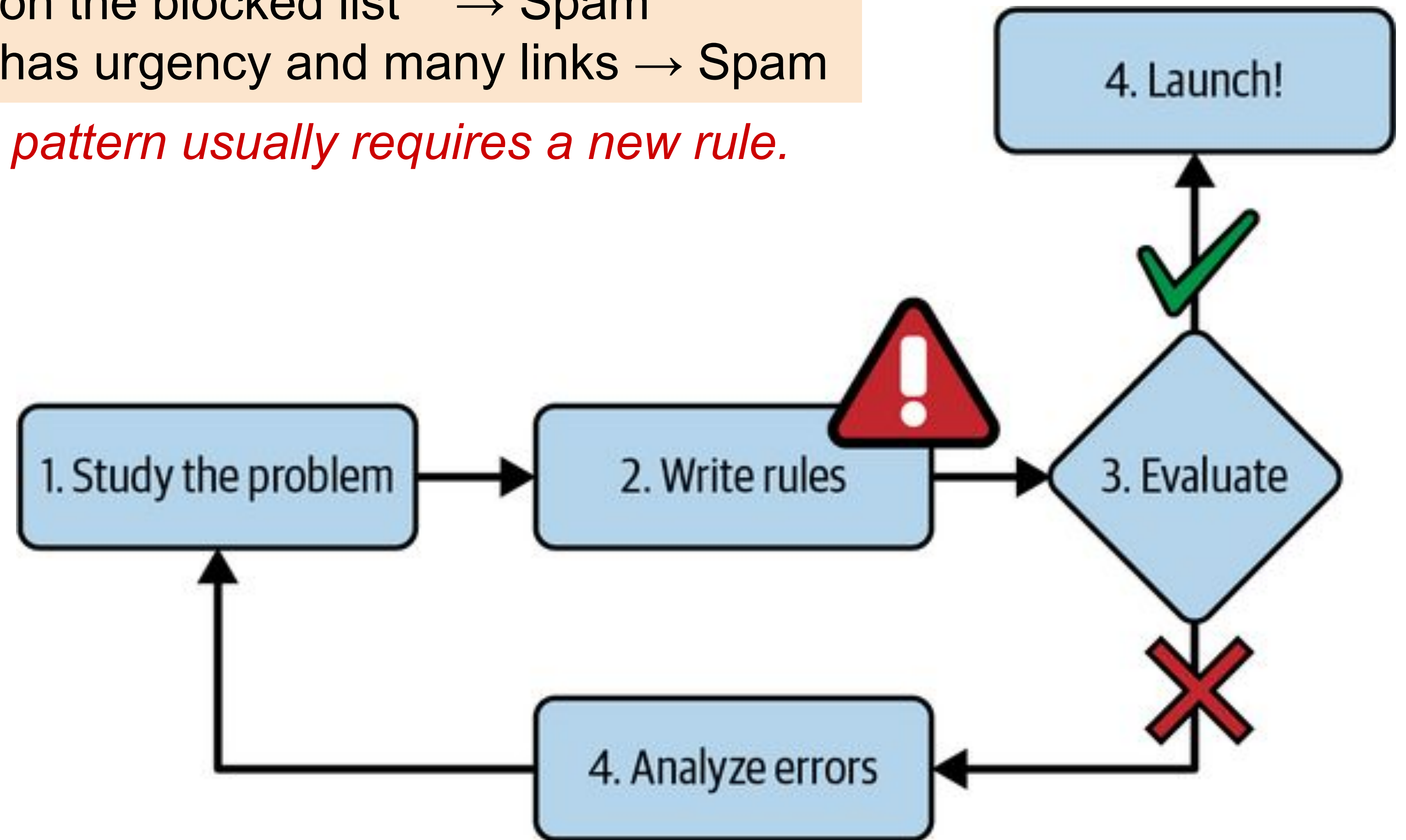
Evaluate Errors



Revise the Rules

IF subject contains “free” → Spam
IF sender is on the blocked list → Spam
IF message has urgency and many links → Spam

A new spam pattern usually requires a new rule.



A Machine Learning Spam Filter

Labeled Emails: Spam and legitimate



Represent Each Email: Words, links, sender, structure



Train a Classifier

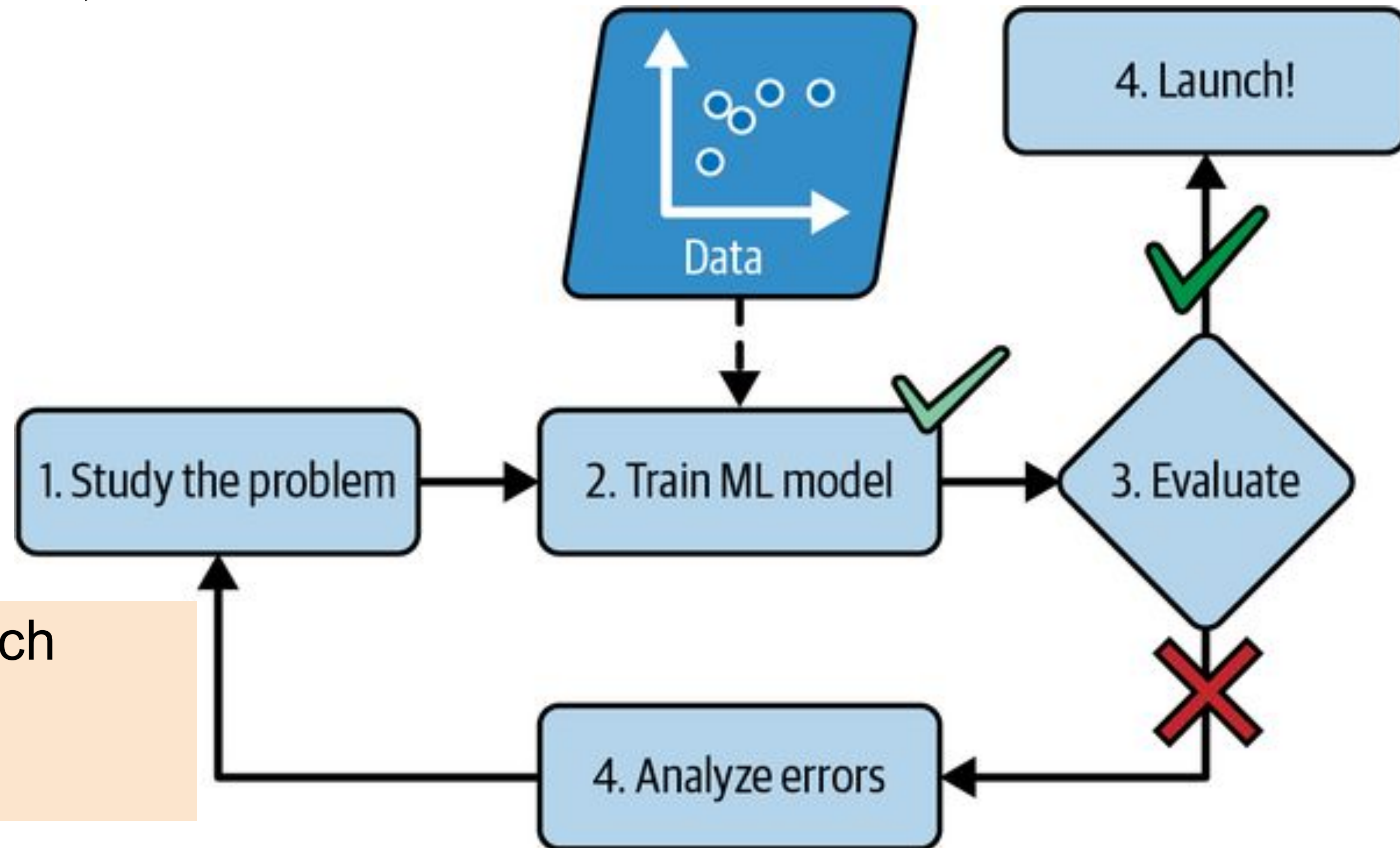


Spam Probability



Spam or Legitimate

The learning algorithm estimates which patterns help distinguish spam from legitimate email.



Traditional vs. Machine Learning

Rule-based system	Machine-learning system
Human specifies patterns	Algorithm estimates patterns
Human revises rules	System is retrained with new examples
Complex rules become difficult to maintain	Data and evaluation guide improvement

Adapting to Changing Data

Incoming Emails



Current Model



Predictions



User Reports and Verified Labels

Deploy Updated Model



Retrain and Evaluate

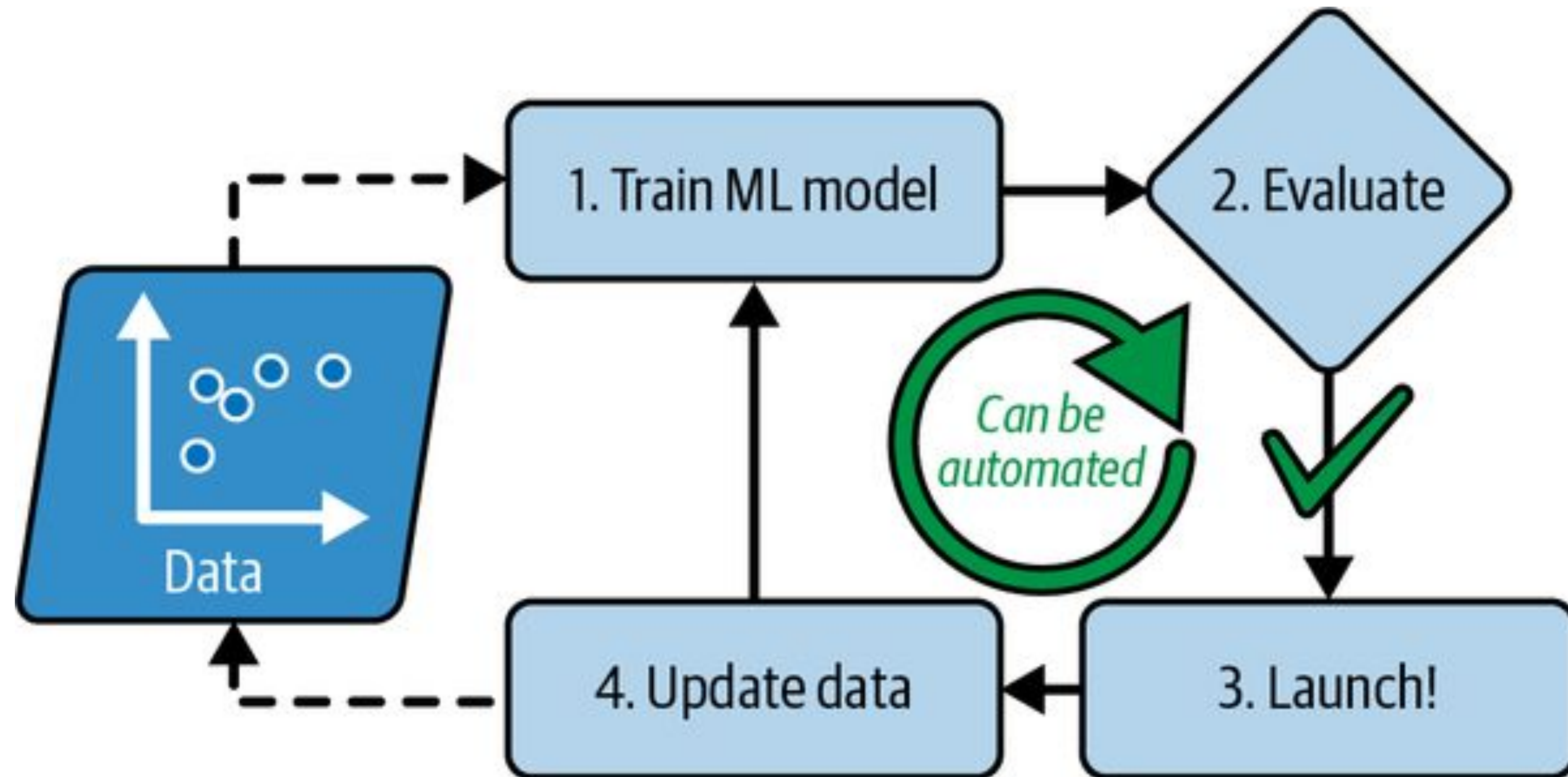


Updated Training

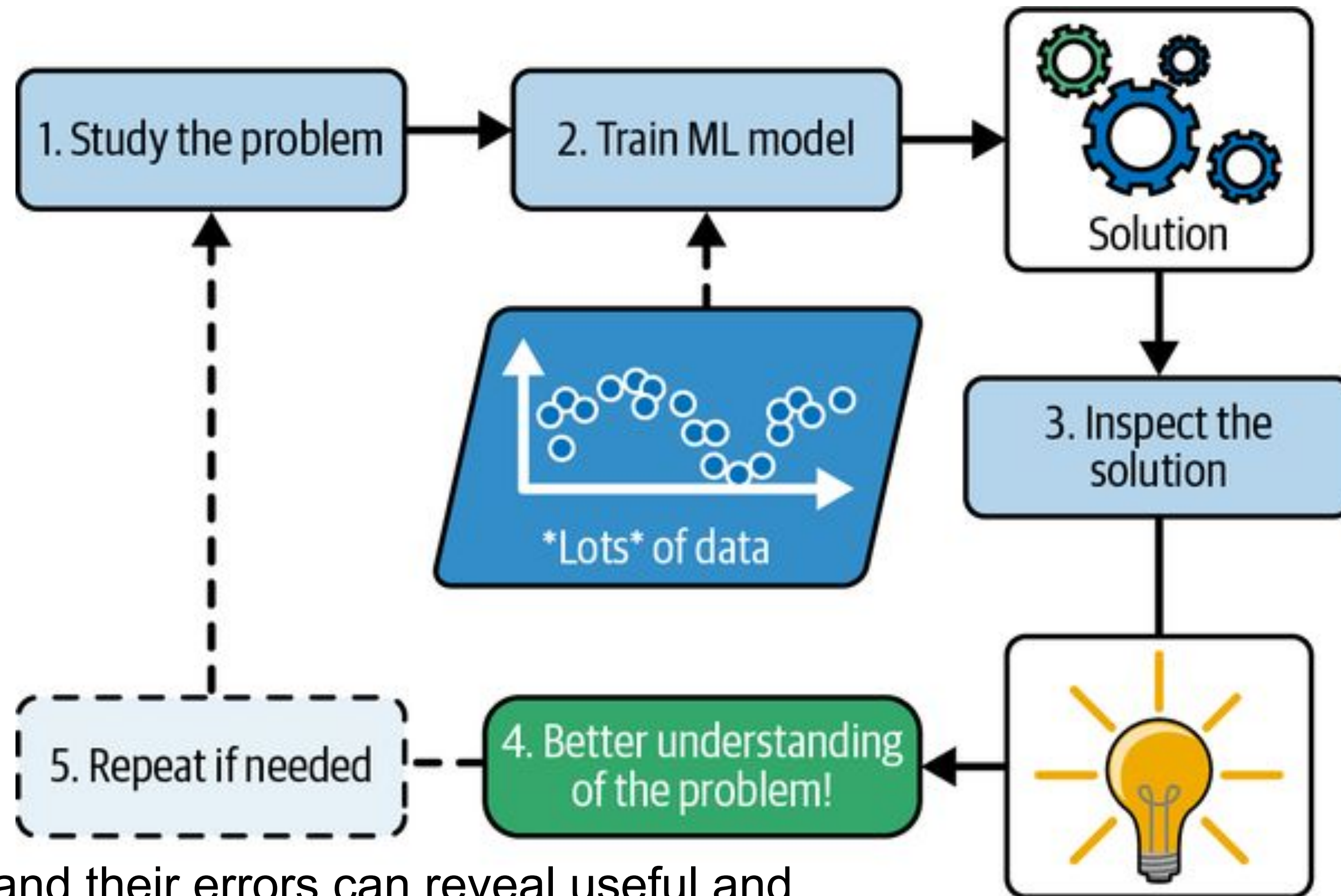


Earlier pattern: “Click to claim your prize”
New pattern: “Tap to receive your reward”

Adaptation requires new evidence, evaluation, and model updates.



Machine Learning Can Reveal Patterns



Inspecting models and their errors can reveal useful and unexpected patterns.

When Machine Learning Helps

- Rules are difficult to specify
 - Speech recognition
 - Image recognition
 - Natural-language understanding
- Patterns change over time
 - Spam detection
 - Fraud detection
 - Product demand
- Predictions depend on complex relationships
 - Disease-risk prediction
 - Weather prediction
 - Equipment-failure prediction

The desired behavior cannot be expressed with a manageable set of rules.

New data may require the model to be updated.

Many variables jointly influence the outcome.

If a small and stable set of rules solves the problem, ordinary programming may be the better choice.

Two More Ways to Describe ML Systems

Dimension	Main question	Categories
Supervision	What feedback is available?	Supervised, unsupervised, reinforcement learning, ...
Model updates	When does the system learn?	Batch or online
Generalization	How does it predict new cases?	Instance-based or model-based

These dimensions can be combined.

Batch vs. Online Learning

Batch learning

- Trains on a collected dataset
- Usually trains offline
- Remains fixed until retraining
- Common in standard ML projects

Online learning

- Updates from arriving examples
- Updates incrementally
- Can adapt between deployments
- Useful for streams or changing data

Batch learning

Collect Data



Train Model Offline



Evaluate



Deploy Fixed Model



Collect New Data



Retrain a New Version

A retailer retrains its demand-forecasting model every Sunday using all available sales data.

The deployed model does not learn until a new version is trained.

Batch Learning Strengths and Limits

Strengths	Limits
Simple training process	Model may become outdated
Easy to test before deployment	Full retraining may be expensive
Easy to version and roll back	Cannot respond immediately
Stable between updates	Requires a retraining schedule

Online Learning: Incremental Updates

New Example or Mini-Batch



Update Model



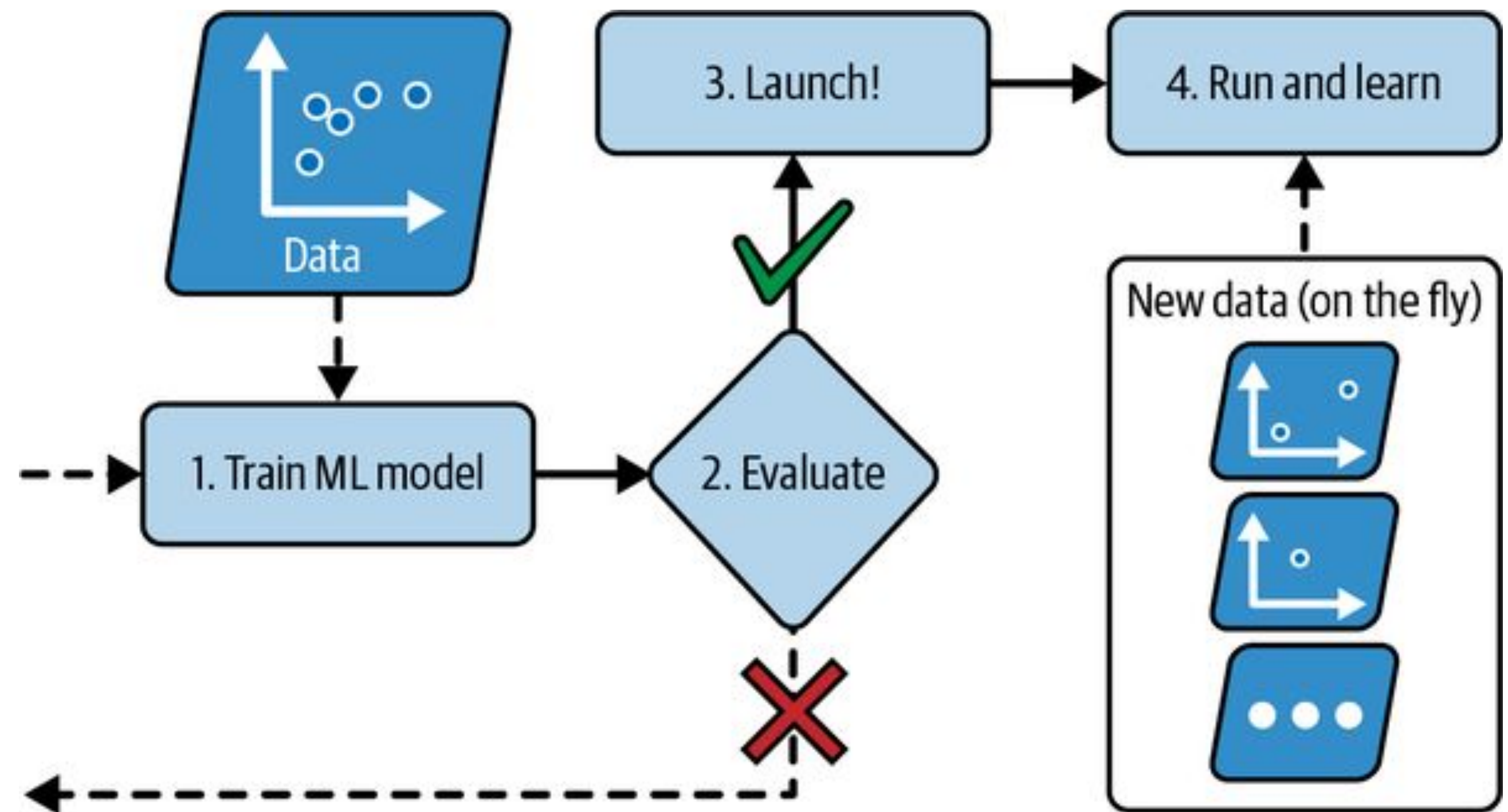
Make Predictions



More New Data

Spam detection
Advertising-response prediction
Sensor monitoring

The model updates without restarting training on the complete dataset.



Online Learning: Adaptation Speed

Stable and slow ← Update rate → Fast and sensitive

Faster adaptation

Responds quickly to change

More sensitive to noise

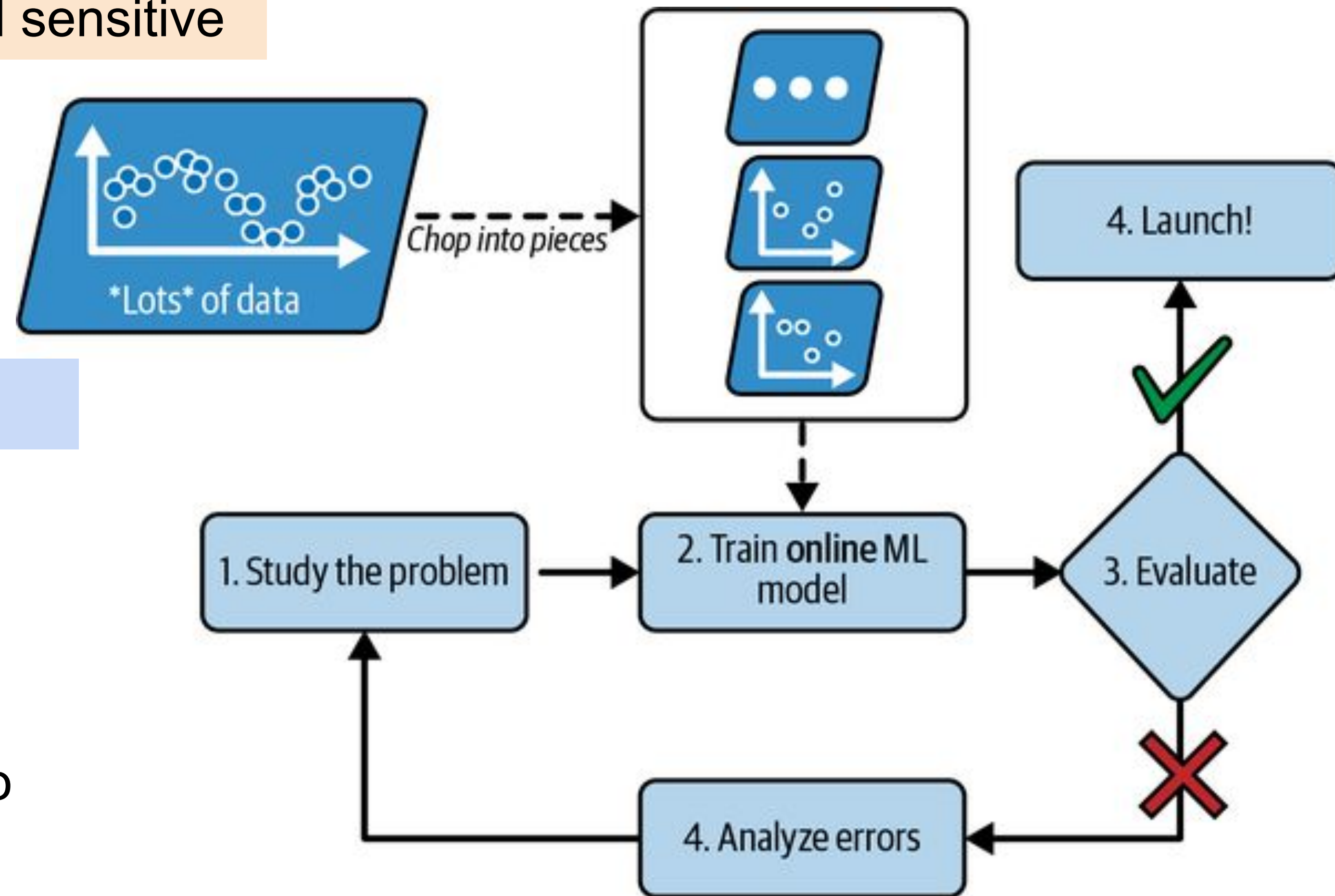
May forget older patterns

Slower adaptation

More stable

Less affected by individual examples

May react too slowly to new patterns



Risks of Online Learning

Bad or Manipulated Data



Immediate Update



Performance Decline

- Monitor inputs
- Monitor performance Pause updates
- Roll back the model Review suspicious data

An online model can learn from mistakes in the data as quickly as it learns from useful examples.



A recommendation model updates immediately from user clicks. A group of users deliberately clicks one product thousands of times. What could happen?

Instance-Based vs. Model-Based Learning

Remember and compare examples

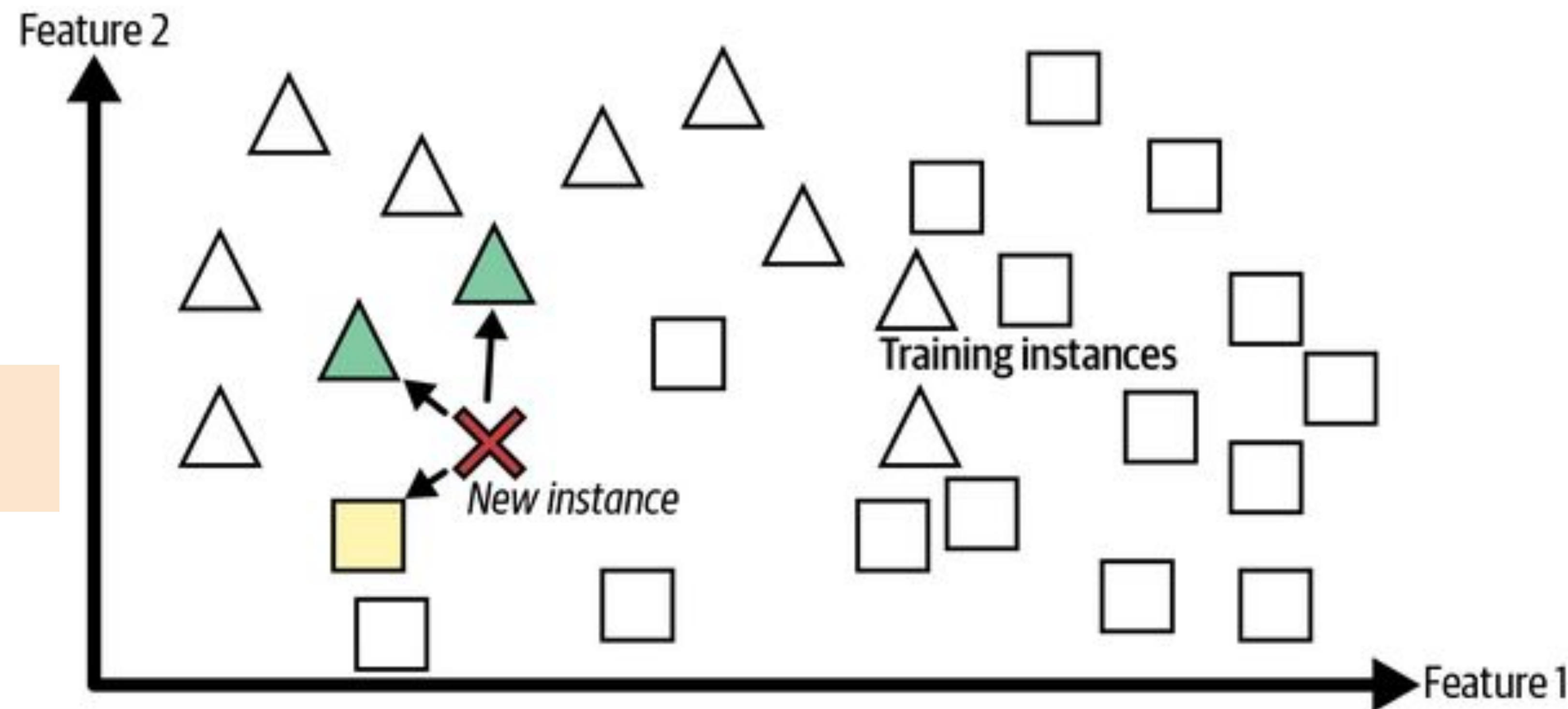


Fit a model and use its parameters.

Instance-based learning

1. Store the training examples
2. Define a similarity or distance measure
3. Find the nearest examples
4. Vote or average their outcomes

Prediction depends on which examples are considered similar.



Model-based learning

Training Examples + Model Family and Loss



Learning Algorithm

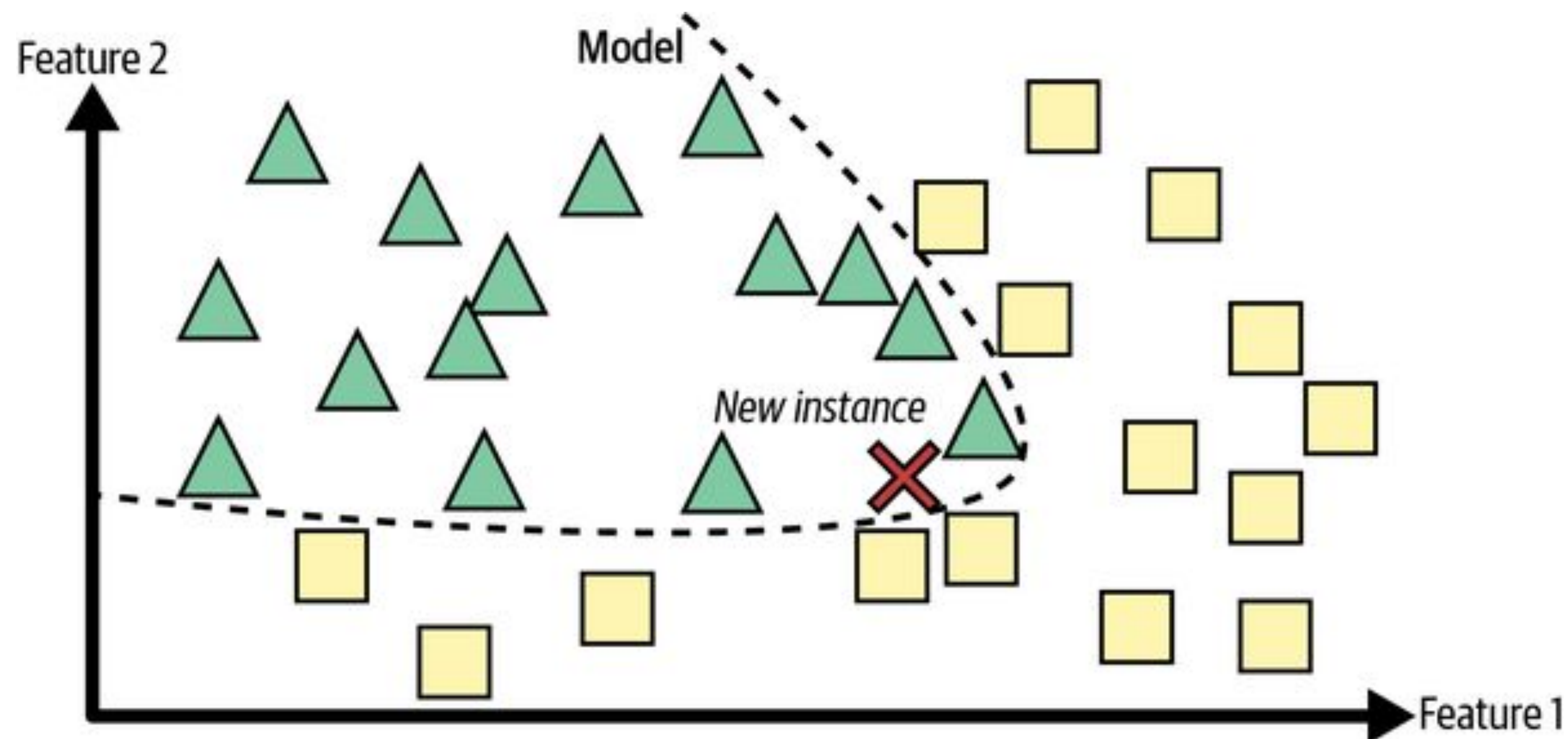


Trained Model Parameters

New Example + Trained Model



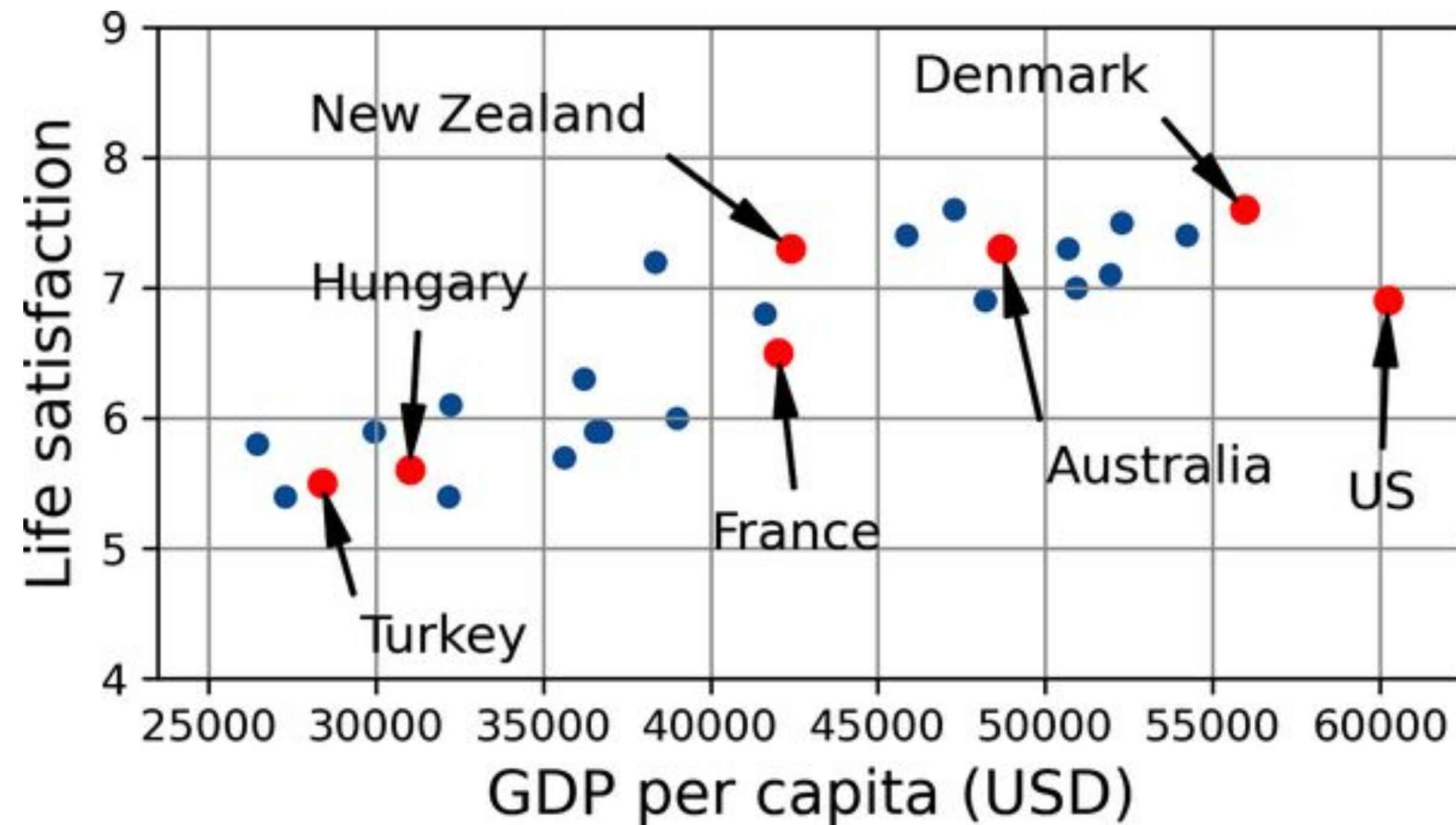
Prediction



Example



Can GDP per capita help predict life satisfaction?

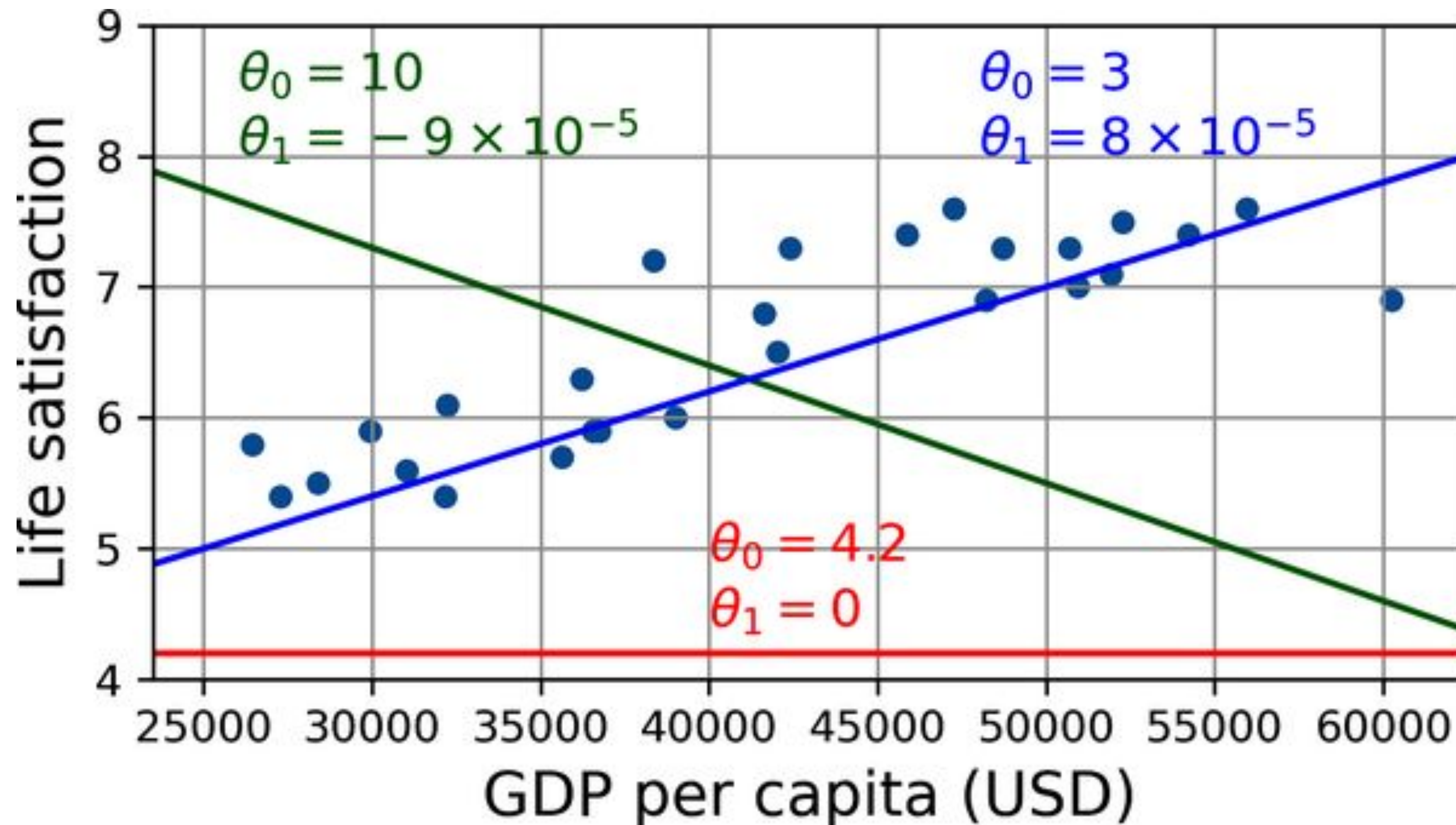


Country	GDP per capita (USD)	Life satisfaction
Turkey	28,384	5.5
Hungary	31,008	5.6
France	42,026	6.5
United States	60,236	6.9
New Zealand	42,404	7.3
Australia	48,698	7.3
Denmark	55,938	7.6

A Model Encodes an Assumption

A simple linear model:

$$\text{life_satisfaction} = \theta_0 + \theta_1 \times \text{GDP_per_capita}$$



Measuring Model Error

$$\text{Loss} = L(y, \hat{y})$$

- The loss compares a prediction with the observed value.
- Training seeks parameter values that reduce the loss.

Model Training

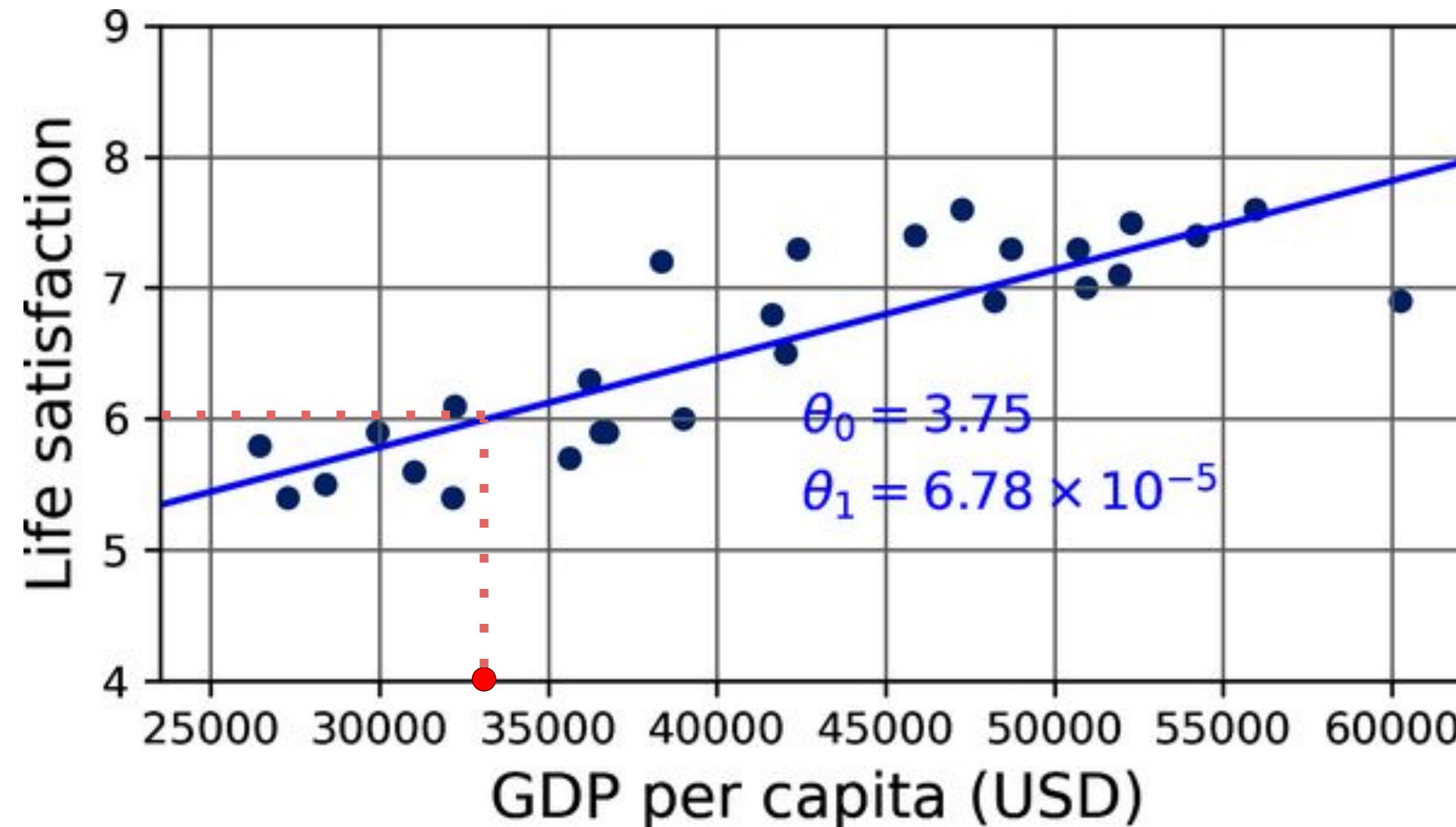
$$\hat{\theta} = \arg \min_{\theta} \frac{1}{n} \sum_{i=1}^n L(f_{\theta}(x_i), y_i)$$

- Model architecture: the form of f_{θ}
- Parameters: values learned during training
- Hyperparameters: choices set outside the training procedure

Example



Puerto Rico's GDP per capita: \$33,442. What is the life satisfaction?



sklearn code example

```
import matplotlib.pyplot as plt
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression

# Download and prepare the data
data_root = "."
lifesat = pd.read_csv(data_root + "lifesat/lifesat.csv")
X = lifesat[["GDP per capita (USD)"]].values
y = lifesat[["Life satisfaction"]].values

# Visualize the data
lifesat.plot(kind='scatter', grid=True,
             x="GDP per capita (USD)", y="Life satisfaction")
plt.axis([23_500, 62_500, 4, 9])
plt.show()

# Select a linear model
model = LinearRegression()

# Train the model
model.fit(X, y)

# Make a prediction for Puerto Rico
X_new = [[33_442.8]] # Puerto Rico' GDP per capita in 2020

print(model.predict(X_new)) # outputs [[6.01610329]]
```

Example

Replacing the linear regression model with *k*-nearest neighbors regression in the previous code is as easy as replacing these lines:

```
from sklearn.linear_model import LinearRegression  
model = LinearRegression()
```

with these two:

```
from sklearn.neighbors import KNeighborsRegressor  
model = KNeighborsRegressor(n_neighbors=3)
```

If all went well, your model will make good predictions. If not, you may need to use more attributes (employment rate, health, air pollution, etc.), get more or better-quality training data, or perhaps select a more powerful model (e.g., a polynomial regression model).

Summary

- You studied the data.
- You selected a model.
- You trained it on the training data.
- Finally, you applied the model to make predictions on new cases (this is called *inference*), hoping that this model will generalize well.

